

PROJECT PROPOSAL

1 Team Details:

1.1 Team Leader:

Name: Harshit Gautam
Roll Number: 250002031
Github: <https://github.com/harshitgautam831>
Discord: <https://discord.com/users/1442431548560511038>
Skills: LTSpice, KiCad, Verilog, C/C++

1.2 Team Members:

1.2.1

Name: Ayusman Dash
Roll Number: 250002017
Github: <https://github.com/ayusmanDash-code>
Discord: <https://discord.com/users/1495815195220508713>
Skills: LTSpice, KiCad, Verilog, C/C++, Java

1.2.2

Name: Anshu Yadav
Roll Number: 250002014
Github: <https://github.com/anshuaasc37-dot>
Discord: <https://discord.com/users/1431962232589123701>
Skills: LTSpice, Verilog, C/C++, Python

1.2.3

Name: Yuvraj Diwakar
Roll Number: 250002087
Github: <https://github.com/yuvrajdiwakar097-ctrl>
Discord: <https://discord.com/users/1495815211222040668>
Skills: LTSpice, Verilog, C/C++

2 Project Specifications:

Domain: Electronics

PS Name: Design and Verification of a High-Speed DDR Display Controller

Preference Number: 1

PROJECT PROPOSAL

3 Project Solution:

- A 50x50 pixel block is to be generated on an FPGA. Registers shall be created to store the x and y coordinates of top left of the block. The direction of movement will be specified by the notations of 1 for moving right/down and 0 for moving left/up. To generate the pixel block, variables 'hcount' and 'vcount' will be created and shall work by the logic: hcount is bounded between box-x and (box-x + 50) and vcount is bounded by similar limits devised with proper logic. We can move once per frame by continuously updating box-x and box-y.
- The intended display resolution is 1920x1080 @ 60 frames/sec. This will require a final clock output (DDR) of 148.5MHz equivalent to 74.25MHz internal CLOCK frequency (SDR). Values are calculated using CEA standards.

Total(for both horizontal and vertical) = Active+Front Porch+Sync+Back Porch

- In SDR to DDR the 2:1 serialization, double the amount of bits are taken at the rising edge of the clock then using this serialization, both the data units are stored (by the help of flip flops) on the rising edge separately. Then, with an inverter, we assign the second data to the falling edge, thereafter using 2 by 1 mux, the clock being its select line, to combine these data in single output. In this way we can spread them across the rising edge (posedge) and falling edge (negedge) giving output at double the clock frequency. BLOCK DIAGRAM is given below

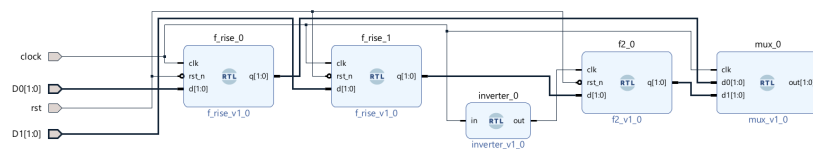


Figure 1: BLOCK DIAGRAM SDR to DDR

- Rather than making serialization of SDR to DDR from basic, We can instantiate ODDR in vivado which is more effective, which is physically located inside the IOB Block of FPGA. This will work the same as the circuit created. We can create testbenches to test the working, time lag etc.

PROJECT PROPOSAL

4 Project Timeline:

- **Week 1:** Drafting a complete planner for step by step implementation and start working on the pixel block first.
- **Week 2:** Coding and implementing the block in Vivado, first implementing just the block without movement then going for the movement part including bouncing off the block from edges.
- **Week 3:** After the block implementation is completed, We will implement the SDR to DDR conversion and ODDR implementation in Vivado.
- **Week 4:** Now after implementation of DDR, we will create test benches, fix time lag, setup/hold time violation and other errors.
- **Week 5:** Finalization and planning for FPGA implementation and analyzing the final output on the monitor.

5 Work Distribution:

- *Harshit Gautam:* Draft the plan, creating the code logic for the pixel block image output then on movement, DDR implementation.
- *Ayusman Dash:* working on the DDR implementation and addressing the fixes of setup/hold time slack report and addressing fixes.
- *Anshu Yadav:* working on the block movement and edge bouncing logic, creating the coordinate tracker, later moving on to the test bench creation to check various aspects of the project at each step.
- *Yuvraj Diwakar:* Working for setup/hold time slack reports, creation of testbench for pixel block movement varification and for DDR implementation, thereafter handling the final FPGA implementation and monitor output.

6 Problem solution - 7458 - HDLbits:

- Harshit Gautam:

PROJECT PROPOSAL

Write your solution here

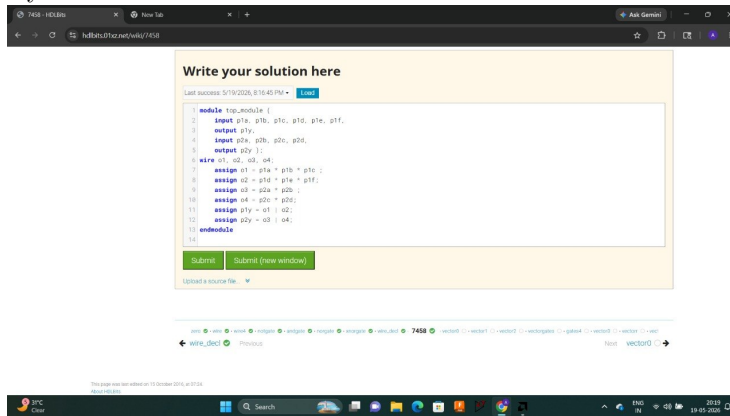
[Load a previous submission]

```

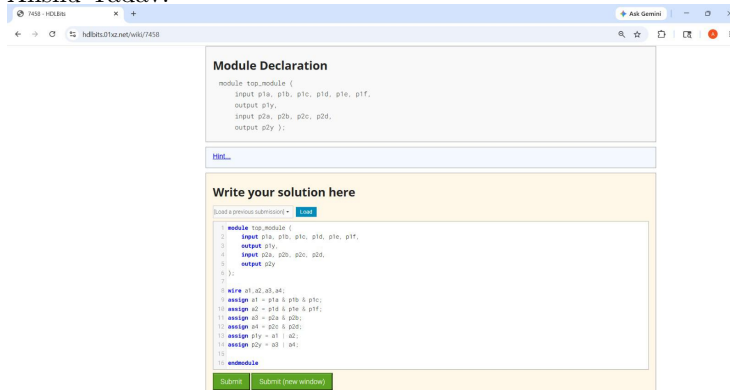
1 module top_module (
2     input wire [2:0] vec,
3     output wire [2:0] outv,
4     output wire o2,
5     output wire o1,
6     output wire o0 ); // Module body starts after module declaration
7     assign o0 = vec[0];
8     assign o1 = vec[1];
9     assign o2 = vec[2];
10    assign outv = vec;
11 endmodule
12

```

• Ayusman Dash:



• Anshu Yadav:



• Yuvraj Diwakar:

PROJECT PROPOSAL

Write your solution here

[Load a previous submission] Load

```
1 module top_module (  
2     input p1a, p1b, p1c, p1d, p1e, p1f,  
3     output p1y,  
4     input p2a, p2b, p2c, p2d,  
5     output p2y );  
6  
7     wire wire_a, wire_b, wire_c, wire_d;  
8  
9     assign wire_a = p1a & p1b & p1c;  
10    assign wire_b = p1d & p1e & p1f;  
11    assign wire_c = p2a & p2b;  
12    assign wire_d = p2c & p2d;  
13  
14    assign p1y = wire_a | wire_b;  
15    assign p2y = wire_c | wire_d;  
16  
17  
18 endmodule  
19
```

Submit Submit (new window)

Upload a source file

7458 — Compile and simulate